

05 — Predict on Test Set

This notebook ties everything together: how to take a trained model and predict on unseen test data.

The golden rule of test prediction:

All statistics (mean , std , U1 , W , w) come from **training data only**. The test set is unseen — you cannot compute anything from it.

Quick decision guide — which pipeline applies?

Question type	Pipeline
Regression, features already 1D	Normalize → design matrix → $\Phi @ W$
Regression, features are 5D (PCA required)	Normalize → PCA project → design matrix → $\Phi @ W$
Logistic regression (classification)	Normalize → add bias → $\text{sigmoid}(X @ w) \geq 0.5$

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
```

Pipeline A — Regression with PCA (exam Problem 1 style)

This is the full exam pipeline:

Load → Normalize → PCA (5D→1D) → Design matrix → Normal Equation → RMSE → Predict test

All steps use training statistics. Test prediction is just the last 3 lines.

```
In [ ]: # =====
# TRAINING PHASE – run once, save everything
# =====

# --- 1(a): Load data ---
# Uncomment and update paths for the exam:
# train_data = np.genfromtxt('train.csv', delimiter=',', skip_header=1)
# test_data = np.genfromtxt('test.csv', delimiter=',', skip_header=1)
# X_train = train_data[:, :5]
```

```

# t_train = train_data[:, 5]
# X_test = test_data

# --- dummy data so this cell runs standalone ---
np.random.seed(0)
base = np.random.randn(50)
X_train = np.column_stack([base*10+15, base*30+50, base*20+35, base*80+130, base*50
t_train = 3*base**2 - 2*base + 1 + np.random.randn(50)*2
X_test = np.column_stack([
    np.random.randn(10)*10+15, np.random.randn(10)*30+50,
    np.random.randn(10)*20+35, np.random.randn(10)*80+130,
    np.random.randn(10)*50+80
])
# -----

print('X_train:', X_train.shape) # (50, 5)
print('t_train:', t_train.shape) # (50,)
print('X_test: ', X_test.shape) # (10, 5)

```

```

In [ ]: # --- 1(b): PCA - reduce 5D to 1D ---

# Step 1: normalize with TRAINING stats
mu_X = X_train.mean(axis=0) # (5,)
sigma_X = X_train.std(axis=0) # (5,)
X_train_norm = (X_train - mu_X) / sigma_X

# Step 2: covariance matrix
m = X_train_norm.shape[0]
Sigma = (1/m) * X_train_norm.T @ X_train_norm # (5, 5)

# Step 3: eigendecomposition
eigenvalues, eigenvectors = np.linalg.eig(Sigma)

# Step 4: sort descending
order = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[order].real
eigenvectors = eigenvectors[:, order].real

# Step 5: project to 1D
U1 = eigenvectors[:, :1] # (5, 1)
x1_train = (X_train_norm @ U1).flatten() # (50,)

print(f'PC1 explains {eigenvalues[0]/eigenvalues.sum()*100:.1f}% of variance')
print('x1_train shape:', x1_train.shape)

```

```

In [ ]: # --- 1(c): Look at the scatter plot, choose degree ---
plt.scatter(x1_train, t_train, color='steelblue', edgecolors='navy', s=40)
plt.xlabel('x1 (PC1)')
plt.ylabel('t')
plt.title('t vs x1 - choose polynomial degree')
plt.grid(True, alpha=0.3)
plt.show()

```

```

In [ ]: # --- 1(d): polynomial regression ---
DEGREE = 2 # <-- set based on the scatter plot above

```

```

def build_design_matrix(x, degree):
    return np.column_stack([x**i for i in range(degree + 1)])

Phi_train = build_design_matrix(x1_train, DEGREE) # (50, 3)
W = np.linalg.inv(Phi_train.T @ Phi_train) @ Phi_train.T @ t_train
t_pred_train = Phi_train @ W
sigma_pred = np.sqrt(np.mean((t_train - t_pred_train)**2)) # RMSE

print('Weights W:', W.round(4))
print(f'Expected uncertainty (RMSE): ±{sigma_pred:.4f}')

# plot
x_line = np.linspace(x1_train.min()-0.1, x1_train.max()+0.1, 300)
t_line = build_design_matrix(x_line, DEGREE) @ W

plt.figure(figsize=(9, 5))
plt.fill_between(x_line, t_line-sigma_pred, t_line+sigma_pred,
                 alpha=0.2, color='red', label=f'±1σ=±{sigma_pred:.2f}')
plt.scatter(x1_train, t_train, color='steelblue', edgecolors='navy', s=40, zorder=5)
plt.plot(x_line, t_line, 'r-', linewidth=2.5, zorder=4, label=f'Degree-{DEGREE} fit')
plt.xlabel('x1 (PC1)')
plt.ylabel('t')
plt.title(f'Polynomial regression degree {DEGREE} - RMSE={sigma_pred:.3f}')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

```

```

In [ ]: # =====
# --- 1(e): PREDICT TEST SET ---
# Apply the SAME pipeline using TRAINING mu_X, sigma_X, U1, W
# =====

# Step 1: normalize test using TRAINING mu_X and sigma_X
X_test_norm = (X_test - mu_X) / sigma_X

# Step 2: project using TRAINING U1
x1_test = (X_test_norm @ U1).flatten()

# Step 3: build design matrix and predict using TRAINING W
Phi_test = build_design_matrix(x1_test, DEGREE)
t_test_pred = Phi_test @ W

print('Predicted values for test set:')
print(f'{"Sample":>8} {"x1":>10} {"t_predicted":>14}')
print('-' * 38)
for i, (xi, ti) in enumerate(zip(x1_test, t_test_pred)):
    print(f'{i:>8} {xi:>10.4f} {ti:>14.4f}')

```

Pipeline B — Regression without PCA (1D input directly)

If the input is already 1D, skip PCA — just normalize and apply polynomial regression.

```
In [ ]: # --- dummy 1D data ---
x_1d_train = np.linspace(0, 5, 40)
t_1d_train = x_1d_train**2 - 2*x_1d_train + np.random.randn(40)
x_1d_test  = np.array([5.5, 6.0, 6.5, 7.0])
# -----

# normalize the 1D feature
mu_1d      = x_1d_train.mean()
sigma_1d   = x_1d_train.std()
x_1d_norm  = (x_1d_train - mu_1d) / sigma_1d

# fit
DEGREE_1D = 2
Phi_1d     = build_design_matrix(x_1d_norm, DEGREE_1D)
W_1d       = np.linalg.inv(Phi_1d.T @ Phi_1d) @ Phi_1d.T @ t_1d_train

# predict test
x_1d_test_norm = (x_1d_test - mu_1d) / sigma_1d # use TRAINING mu and sigma
Phi_1d_test    = build_design_matrix(x_1d_test_norm, DEGREE_1D)
t_1d_test_pred = Phi_1d_test @ W_1d

print('Predictions for 1D test:')
for i, (xi, ti) in enumerate(zip(x_1d_test, t_1d_test_pred)):
    print(f' x={xi:.1f} → t_pred={ti:.4f}')
```

Pipeline C — Logistic Regression prediction on test set

```
In [ ]: # Assume you trained w, mu_cls, std_cls as in notebook 04
# These are stand-ins:
w_trained = np.array([[0.5], [2.0], [1.5]])
mu_train  = np.array([0.0, 0.0])
std_train = np.array([1.0, 1.0])

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# X_test_cls: shape (m_test, 2) – raw feature values
X_test_cls = np.array([[0.5, 0.3], [-1.0, -0.5], [1.2, 0.8]])
m_test_cls = X_test_cls.shape[0]

# Step 1: normalize with TRAINING stats
X_test_cls_norm = (X_test_cls - mu_train) / std_train

# Step 2: add bias
X_test_cls_bias = np.hstack([np.ones((m_test_cls, 1)), X_test_cls_norm]) # (m, 3)

# Step 3: predict
h_test = sigmoid(X_test_cls_bias @ w_trained) # probabilities (m, 1)
```

```

y_pred      = (h_test >= 0.5).astype(int)           # binary class

print('Test predictions:')
for i in range(m_test_cls):
    print(f' Sample {i}: prob={float(h_test[i]):.4f} class={int(y_pred[i])}')

```

What NOT to do on test data

```

# WRONG - recomputing normalization from test data
mu_wrong      = X_test.mean(axis=0)                # ← NO
sigma_wrong   = X_test.std(axis=0)                 # ← NO
X_test_norm   = (X_test - mu_wrong) / sigma_wrong  # ← WRONG

# CORRECT - always use training statistics
X_test_norm   = (X_test - mu_X) / sigma_X          # ← YES, mu_X from
training

# WRONG - running PCA again on test
eigenvalues2, eigenvectors2 = np.linalg.eig(...)  # ← NO, don't redo this

# CORRECT - reuse training eigenvectors
x1_test      = (X_test_norm @ U1).flatten()       # ← YES, U1 from
training

```

Checklist before submitting exam

- ☐ Every cell labelled: # --- 1(a) --- , # --- 1(b) --- etc.
- ☐ PCA: normalize → covariance → eig → sort → project
- ☐ `.real` added after `np.linalg.eig`
- ☐ Sort uses `eigenvectors[:, order]` (columns!) not `eigenvectors[order]`
- ☐ `mu_X`, `sigma_X`, `U1` used on test (not recomputed)
- ☐ Normal equation: $W = \text{inv}(\Phi.T @ \Phi) @ \Phi.T @ t$
- ☐ RMSE = `sqrt(mean((t - t_pred)**2))`
- ☐ Test predictions printed clearly